

PyQCU 纯 Python 多重网格算法实现

层级构建、Galerkin 粗算子、BiCGStab 平滑与证据化核验

代码审计与算法报告

PyQCU

2026-08-28

结论先行：V-cycle 骨架完整，但粗层算子存在可复现语义断裂

已确证

层级尺寸、近零模、局部块 QR、 $P^\dagger/P$ 转移、BiCGStab 平滑、递归粗修正与 R3 状态重置均有明确代码路径。

核验结果

单层 4^4 Wilson CPU 实测真实残差 4.81×10^{-11} ；转移往返误差 8.88×10^{-16} 。

首要行动

粗层已构造的 M 被 `clover_term is None` 分支绕过；另需补齐 s 提前收敛与外部 $x^{(0)}$ 接入。

当前判断： **确证** 细层 BiCGStab 与层间转移具备可运行实现；**风险** 纯 Python 多层 Galerkin 算子的“构造式”与“应用式”目前不一致，因而不能把多层收敛性归因于已正确实现的粗空间。

来源：solver/_multigrid.py:353-540；dslash/_operator.py:157-226；本会话 CPU 只读 smoke 实验。

任务定义与证据边界

任务定义

面向学者和后续开发者，解析 solver/_multigrid.py 的纯 Python 多重网格实现：说明物理对象、数学算子、层级工作流、停止语义与当前证据；不修改源代码。

范围	角色与分析内容	状态
solver/_multigrid.py	主入口、层级管理、V-cycle、BiCGStab、adaptive、solve	确证 精读
tools/_multigrid.py	近零模、局部 QR、restrict/prolong	确证 主题函数精读
dslash/_operator.py	细层算子、Galerkin 粗算子、Wilson/Clover sitting	确证 直接依赖精读
dslash/_wilson.py	hopping 矩阵与邻居作用方向	确证 直接依赖精读
tools/_linalg.py、 tools/_define.py、 cann/__init__.py	MPI 内积、范数、布局、NPU 兼容	确证 相关段落
CUDA 内核、示例、日志、全仓库 benchmark	非纯 Python 实现或超出本次限定边界	未验证 未覆盖

来源：范围来自用户指定的 solver/_multigrid.py；局部 AGENTS 约定已读取。

主体结构：求解器由五个可追踪层次组成

层次	责任	关键证据
求解入口	维护参数、层级尺寸、右端项与生命周期，输出 $[4, 3, L_x, L_y, L_z, L_t]$	<code>solver/_multigrid.py:10-89, 542-557</code>
V-cycle 控制	当前层 BiCGStab；周期性残差限制、粗层递归、细层校正	<code>solver/_multigrid.py:353-540</code>
层间表示	随机近零模 $\rightarrow$ 局部正交基 $\rightarrow R/P$	<code>solver/_multigrid.py:270-306;</code> <code>tools/_multigrid.py:8-139</code>
Dirac 算子	$D = S + H$; hopping 处理四个方向，sitting 处理局部项	<code>dslash/_operator.py:9-147;</code> <code>dslash/_wilson.py:230-324</code>
全局数值支持	MPI Allreduce 的 $v^\dagger w$ 与全局范数；NPU 复数拆分兼容	<code>tools/_linalg.py:7-41;</code> <code>cann/_init__.py:7-41, 55-140</code>

来源：代码调用摘要；主体路径为纯 Python (`solver/_multigrid.py:353-540`)。

各部分关系：构建链与求解链在同一对象上汇合

输入： $U, C, \kappa, u_0, \{d_i\}, \{g_\mu\}$, dtype/device

↓

细层： $D_0 = S_0 + H_0$, 由 `dslash.operator` 组合 Wilson/Clover 作用

↓

每个层间接口： $V_i \xrightarrow{\text{local QR}} P_i, \quad R_i = P_i^\dagger, \quad D_{i+1} \xleftarrow{\text{Galerkin}} P_i^\dagger D_i P_i$

↓

求解： $\text{solve} \rightarrow \text{cycle}(0) \rightarrow \text{BiCGStab smoothing} \rightarrow R_i r_i \rightarrow \text{cycle}(i+1) \rightarrow P_i e_{i+1}$

↓

输出： x_0 的展平内部表示 $[12, \text{xyzt}] \rightarrow$ 返回费米子布局 $[4, 3, \text{xyzt}]$ 。

来源： `solver/_multigrid.py:223-307, 353-540`; `dslash/_operator.py:157-226`。箭头表示真实调用/数据依赖。

项目思路：以低模近零空间压缩长程误差

为什么使用近零模

细层迭代器对高频误差通常容易削弱，难点是低频误差。迭代式 $v \leftarrow v - A^{-1}Av$ 用宽松相对容差保留慢变分量，再把这些分量组织成粗空间。

为什么局部 QR

每个粗格点对应一个细块；块内 QR 使 $P_i^\dagger P_i \simeq I$ ，保证限制/延拓不会因基向量尺度漂移而放大误差。

为什么 Galerkin

$D_{i+1} = P_i^\dagger D_i P_i$ 将细层物理算子投影到粗空间，保留规范场、spin-color 耦合和局部/邻接结构。

为什么自适应回退

若窗口中当前残差比过半历史值更差，累计停滞计数；连续达到阈值后停止继续进入粗层，避免无效粗修正。

时间	代码演进证据	对当前实现的影响
2026-07-28	R2 breakdown 检测	数值奇异时自动重启，保留 x, r
2026-08-14	CUDA 同步、资源释放、R3 状态重置	混合路径生命周期更明确
2026-08-24	seed 与多实例资源约定	随机初始化可复现，但不改变粗算子语义

来源：演进旁证：tools/_multigrid.py:29-42; solver/_multigrid.py:407-449; 设计动机标注为 **推断**。

对象映射：代码张量对应格点 QCD 中的什么

代码对象	数学/物理对象	布局或作用
U	规范链变量 $U_\mu(x) \in SU(3)$	[3,3,4,Lx,Ly,Lz,Lt]，四个方向平移
D/S/H	Clover/Wilson Dirac 算子、局部项、hopping 项	$D = S + H$; H 连接相邻格点
null_vecs	近零空间候选向量	$[E_i, E_{i-1}, x, y, z, t]$ ，逆迭代后归一化
lonv/P	每个粗块的局部正交基/延拓	$[E_i, E_{i-1}, X, x, Y, y, Z, z, T, t]$
restrict/prolong	$P^\dagger$ 限制 / P 延拓	粗残差压缩、粗修正返回细层
r_tilde,p,v,s,t	BiCGStab 的影子残差、搜索/像向量	每层独立维护，粗修正后全部重置
b_list[level]	当前层方程右端项	细层为原始 b ，粗层为 $P^\dagger r$

来源：solver/_multigrid.py:19-79,392-400; tools/_multigrid.py:69-139;
dslash/_wilson.py:10-18,230-264。

层级构建 (1/2): 由格点尺寸和自由度生成层次

Input: $L^{(0)} = (L_x, L_y, L_z, L_t)$, 最小尺寸 m , 层数上限 K , 粗化因子 $g = (g_x, g_y, g_z, g_t)$, 每层自由度 E_i 。

Initialize: $\text{lat_size_list} \leftarrow []$, $\ell \leftarrow L^{(0)}$ 。

while: $\min_{\mu} \ell_{\mu} \geq m$ 且已加入层数 $< K$:

$\text{lat_size_list.append}(\ell)$ 。

$\ell_{\mu} \leftarrow \lfloor \ell_{\mu} / g_{\mu} \rfloor$, $\mu \in \{x, y, z, t\}$ 。

Set state: $N_{\text{level}} \leftarrow |\text{lat_size_list}|$, $\text{dof/dtype/device} \leftarrow$ 对应前缀切片。

Sanity condition: ℓ_{μ} 必须可被局部块尺寸整除, 否则局部正交化的 reshape/assert 不能成立。

Pure-Python alignment: $g = (2, 2, 2, 2)$ 与粗算子构建器的偶/奇坐标探针假设一致。

Output: $L^{(0)} \rightarrow L^{(1)} \rightarrow \dots \rightarrow L^{(N-1)}$ 及每层算子容器。

来源: solver/_multigrid.py:34-45; tools/_multigrid.py:69-85; dslash/_operator.py:157-166。

层级构建 (2/2): 近零模到局部正交粗空间

For $i = 1, \dots, N-1$: 生成随机候选 $V_i \in \mathbb{C}^{E_i \times E_{i-1} \times L^{(i-1)}}$ 。

Inverse iteration: $v_{i,k} \leftarrow v_{i,k} - A_{i-1}^{-1}(A_{i-1} v_{i,k})$, 其中 A^{-1} 由 BiCGStab 近似实现, 相对容差为 5×10^{-5} 。

Normalize: $v_{i,k} \leftarrow v_{i,k} / \|v_{i,k}\|$ 。

Block reshape: $[E_i, E_{i-1}, Xx, Yy, Zz, Tt] \rightarrow [X, Y, Z, T, E_i, E_{i-1}, x, y, z, t]$ 。

Local QR: $A_{\text{block}} \in \mathbb{C}^{(E_{i-1} \times yzt) \times E_i}$, $A_{\text{block}} = Q_{\text{block}} R_{\text{block}}$ 。

Restore: $Q_{\text{block}} \rightarrow P_i[E_i, E_{i-1}, X, x, Y, y, Z, z, T, t]$ 。

Construct: $D_i \leftarrow P_i^\dagger D_{i-1} P_i$ 的代码探针并入 `op_list`。

End for. 保存 `nv_list`、`lonv_list`、`b_list` 与 `op_list`。

来源: `solver/_multigrid.py:270-306`; `tools/_multigrid.py:8-66, 69-109`。

转移算子：局部正交性使限制与延拓成为伴随对

数学定义

对局部正交基 P_i :

$$(R_i v)_{E,X,Y,Z,T} = \sum_{e,x,y,z,t} P_i^\dagger{}_{E,e,Xx,Yy,Zz,Tt} v_{e,Xx,Yy,Zz,Tt},$$

$$(P_i c)_{e,Xx,Yy,Zz,Tt} = \sum_E P_{iE,e,Xx,Yy,Zz,Tt} c_{E,X,Y,Z,T}.$$

因此 $R_i = P_i^\dagger$ ，理想情况下 $R_i P_i = I$ 。

物理含义：延拓把粗格点自由度展开为细格点上的低频场形状；限制用共轭转置把细层残差投影回同一低频子空间。

来源：tools/_multigrid.py:112-139；tools/_multigrid.py:86-105；实测命令：CPU、float64、随机候选、固定 seed=7。

CPU/float64 受控核验

细格点： 4^4 ；粗格点： 2^4 ； $E_i = 2$ 、 $E_{i-1} = 1$ 。

$$\max |P^\dagger P - I| = 4.44 \times 10^{-16},$$

$$\max |R(Pc) - c| = 8.88 \times 10^{-16}.$$

确证 转移路径数值自治。

Galerkin 粗算子：构造公式正确，当前纯 Python 应用分支未闭合

阶段	代码行为	数学意义
初始化	sitting.M 与四个方向的 M_plus/M_minus 置零	为 $E_i \times E_i$ 粗矩阵分配空间
探针	对每个粗自由度、每个方向构造粗基向量并 prolong	计算 $P_i e_a$
细层作用	分别应用 fine_hopping.matvec_plus/minus	得到 $H_{i-1} P_i e_a$
回投影	restrict 细层结果	得到 $P_i^\dagger H_{i-1} P_i e_a$
归类	同块部分写入 sitting.M, 跨块部分写入 hopping	保留粗层局部项与邻接项
细层局部项	fine_sitting.matvec 回投影并累加到 sitting.M	得到完整 $P_i^\dagger D_{i-1} P_i$
应用分支	sitting.matvec 在 clover_term is None 时直接返回输入	绕过已构造的粗层 M

风险 可复现证据：纯 Wilson、 $U = I$ 、 $4^4 \rightarrow 2^4$ 、 $E_i = 2$ 时，粗层 M-I 范数为 0.160529；默认应用与 $P_i^\dagger D P_i$ 最大差为 0.0564071。临时让 sitting.matvec 使用构造出的 M 后，差为 9.93×10^{-16} 。

来源：dslash/_operator.py:157-226；dslash/_operator.py:135-147；solver/_multigrid.py:305-306；本会话 Galerkin 对照实验。

V-cycle 核心 (1/2): 每层先执行 BiCGStab 平滑

Input: 当前层算子 A_ℓ 、右端 b_ℓ 、平滑容差 `_tol`、最大迭代数。

Initialize: $x \leftarrow 0$, $r \leftarrow b_\ell - A_\ell x$, $\tilde{r} \leftarrow r$; $p, v, s, t \leftarrow 0$, $\rho_{\text{prev}}, \alpha, \omega \leftarrow 1$ 。

for $i = 0, 1, \dots$:

$\rho_i \leftarrow \tilde{r}^\dagger r$; 若 $|\rho_i| < 10^{-20}$, 进入自动重启。

$\beta_i \leftarrow (\rho_i / \rho_{\text{prev}})(\alpha / \omega)$; $p \leftarrow r + \beta_i(p - \omega v)$ 。

$v \leftarrow A_\ell p$; 若 $|\tilde{r}^\dagger v| < 10^{-20}$, 进入自动重启。

$\alpha \leftarrow \rho_i / (\tilde{r}^\dagger v)$; $s \leftarrow r - \alpha v$ 。

Expected early branch: **if** $\|s\| \leq \text{tol}$ **then** $x \leftarrow x + \alpha p$ **and stop**。 **风险** 当前代码未实现。

$t \leftarrow A_\ell s$; 若 $|t^\dagger t| < 10^{-20}$, 进入自动重启。

来源: `solver/_multigrid.py:376-449`; 用户提供的 BiCGStab 伪代码 (书目信息未提供, 仅作参考/算法参照)。

V-cycle 核心 (2/2): 残差更新、粗修正与状态重置

$\omega \leftarrow (t^\dagger s)/(t^\dagger t)$; 若非有限, 进入自动重启。

$x \leftarrow x + \alpha p + \omega s$, $r \leftarrow s - \omega t$ 。

记录细层残差; 若 $\|r\| < \text{_tol}$, 本层返回。

`count_restart += 1`; 当 $\ell < N - 1$ 且 `count_restart > num_restart` 时启动粗网格周期。

Cycle start: $r_{\ell+1} \leftarrow R_\ell r_\ell$, 写入 `b_list[level+1]`。

$e_{\ell+1} \leftarrow \text{cycle}(\text{level}+1)$; $e_\ell \leftarrow P_\ell e_{\ell+1}$ 。

$x_\ell \leftarrow x_\ell + e_\ell$; $r_\ell \leftarrow b_\ell - A_\ell x_\ell$ (重新计算真残差)。

R3 reset: $\tilde{r} \leftarrow r$, $p, v, s, t \leftarrow 0$, $\rho_{\text{prev}}, \alpha, \omega \leftarrow 1$ 。

Stop: $\|r_\ell\| < \text{_tol}$ 或达到 `max_iter`; 否则回到下一次平滑。

Output: 当前层 x_ℓ ; 层 0 再恢复为 `[4,3,xyzt]`。

来源: `solver/_multigrid.py`:450-540; R3 状态重置证据: `solver/_multigrid.py`:487-501。

停止语义与自适应回退：代码中的“收敛”不是单一判据

位置	判据/状态变更	解释
初始检查 BiCGStab 循环	$\ r_0\ < \text{tol}$ (层 0) 或 $0.5\ r_0\ /0.1\ r_0\ $ (粗层) $\ r_i\ < _tol$	粗层只做相对初始残差的内部求解 当前层返回；层 0 的 $_tol$ 为用户绝对 tol
自适应窗口 层级回退 求解复位	当前残差大于等于窗口中至少一半历史改善值 $\text{convergence_tol} > 3$ $\text{levels_back}()$ 只复位计数和层数	$\text{convergence_tol} += 1$ ，表示停滞 将 num_level 置为 1，不再进入粗层 $\text{convergence_history}$ 当前未清空， 重复 solve 会沿用旧历史
初值接口	$\text{solve}(x0=...)$ 保存到 self.x0	cycle 重新创建零初值；当前代码未 消费 self.x0

风险 语义边界：代码打印的“Final residual”是递推残差；单层实测另外计算的真残差与其一致，但多层/低精度场景仍应显式做 $\|b - Dx\|$ 复核。

来源：`solver/_multigrid.py:338-351,376-390,502-517,542-557`。

物理与极限检查：应保持的结构不变量

检查	预期不变量	当前证据
规范协变结构	hopping 保持 $(1 \mp \gamma_\mu)U_\mu$ 与反向 $(1 \pm \gamma_\mu)U_\mu^\dagger$ 的方向配对	确证 dslash/_wilson.py: 10-18, 230-264
局部性	单个粗块只连接自身及四方向邻居；同块项进入 sitting	确证 构造代码存在，应用分支有风险
粗空间正交	$P^\dagger P = I$ 块内成立	确证 float64 误差 4.44×10^{-16}
零 hopping 极限	$\kappa \rightarrow 0$ 时 $D \rightarrow I$ ，粗算子应退化为单位	推断 能掩盖当前内部 hopping 丢失，不能证明有限 κ 正确
奇偶预处理	Schur 算子应在粗转移前后保持同一 parity 语义	未验证 当前代码在粗限制前重建 full residual，需专门对照
量纲/尺度	residual 范数与 tol 同为无量纲离散线性系统量	确证 代码只比较范数；物理格距未进入该接口

来源：dslash/_wilson.py:10-18, 230-264; solver/_multigrid.py:360-375, 462-485; 物理极限结论中标注 推断 / 未验证。

实测证据：单层正确运行，多层证据暴露粗层缺口

ID	设置与命令性质	实际输出	状态
C1	CPU、Wilson、 $U = I$ 、 4^4 、 dof=[12]、tol=1e-10、 max_iter=30，调用 solve	返回 [4,3,4,4,4,4]；真实残差 $4.8097878 \times 10^{-11}$ ；history 长度 49	确证
C2	CPU、float64、随机局部基， $4^4 \rightarrow 2^4$ ，检查 QR 与 restrict(prolong(c))	$\max P^\dagger P - I = 4.44 \times 10^{-16}$ ； $\max RPc - c = 8.88 \times 10^{-16}$	确证
C3	CPU、float64、Wilson Galerkin 对 照， $E_i = 2$	默认 $\max D_c c - RD_f P c = 5.6407137 \times 10^{-2}$ ； 临时使用 sitting.M 后 $9.9301366 \times 10^{-16}$	确证 风险定位
C4	两层 init/solve， $4^4 \rightarrow 2^4$ 、 dof=[12,1]、max_iter=12、 tol=1e-4	初始化完成；最终真实细层残差 1.4218765×10^{-3} ，未达设定容差	未验证 未通过

来源：C1–C4 均为本会话在 MPI size=1、CPU 上执行的只读 Python 实验；C3 还使用了仅在内存中改变分支条件的诊断对照。

缺陷与改进优先级：先恢复数学闭合，再扩展验证

优先级	观察到的实现问题	影响	建议验收
P0	粗层 <code>sitting.M</code> 已含同块 <code>hopping</code> ，但 <code>clover_term</code> is <code>None</code> 直接返回输入	$D_c \neq P^\dagger D_f P$ ，多层校正方程改变	修复后 C3 最大差 $\leq 10^{-12}$ (float64)
P1	s 足够小时未提前接受 $x + \alpha p$; $t = 0$ 被当作 <code>breakdown</code>	精确/幸运收敛可能反复重启; <code>identity</code> 受控例触发 <code>ZeroDivisionError</code>	<code>identity</code> 、对角矩阵与随机矩阵测试均无异常且残差达标
P1	<code>solve(x0)</code> 只写 <code>self.x0</code> , <code>cycle</code> 固定零初值	热启动接口名存实亡	非零 $x^{(0)}$ 的初始残差与结果可观测改变
P2	<code>convergence_history</code> 未在新 <code>solve</code> 开始时清空	重复求解的 <code>adaptive/plot</code> 历史混合	两次 <code>solve</code> 的 <code>history</code> 分段独立
P2	<code>except RuntimeError</code> 覆盖整个数值块	后端真正运行时错误可能被误报为 <code>breakdown</code>	只捕获显式 <code>breakdown</code> 类型/标志
P2	<code>mg_grid_size</code> 可配置，但粗算子构造注释和探针按 2 粗化	非标准粗化因子可能错位	非 2 因子下 Galerkin 对照通过或明确拒绝

来源: `solver/_multigrid.py:411-449`; `dslash/_operator.py:157-226`; `identity` 受控实验: `ZeroDivisionError`。

下一步：可验收的最小修复—验证闭环

阶段	目标	可验收产物/判据	当前状态
1	修复粗层 sitting 应用语义	在粗算子对象中区分“细层纯 Wilson 的 identity”与“已构造 Galerkin 的 M”；C3 误差 $\leq 10^{-12}$	待实现
2	对齐 BiCGStab 参考伪代码	增加 $\ s\ $ 早停；breakdown 不再吞后端异常；identity test 不抛除零	待实现
3	接通热启动和历史状态	x0 真正进入 cycle；每次 solve 清空或分段记录历史	待实现
4	回归数学关系	$RP = I$ 、 $D_c = P^\dagger D_f P$ 、full residual 三项对照	本报告已提供 基线
5	物理/并行扩展	Wilson/Clover、parity、MPI size>1、不同 dtype 的真残差与收敛曲线	未验证 未执行

来源：行动项由 C1-C4 证据和 solver/_multigrid.py:353-557 推出；非本次任务授权范围内未修改代码。

分析框架 A1–A8：从目的到实际输出

步骤	本报告结论	证据状态/入口
A1 目的与前置	用层间粗空间削弱低频误差；需 PyTorch、MPI、算子与布局约定	确证 solver/_multigrid.py:10-58
A2 输入	U, C, κ, u_0 、层级尺寸、dtype/device、tol/max_iter、可选 parity	确证 solver/_multigrid.py:11-45
A3 输出	求解场 [4,3,xyzt]，并保留收敛历史/可选图	确证 solver/_multigrid.py:542-576
A4 整体框架	init 建层，solve 进入 cycle，各层用同一 BiCGStab 结构	确证 solver/_multigrid.py:223-307,353-540
A5 关键细节	逆迭代、局部 QR、 R/P 、Galerkin 探针、R3 reset	确证 相关代码已逐段精读
A6 正确执行	构造 $\rightarrow$ init() $\rightarrow$ solve()；多层必须先准备 lonv/op_list	确证 solver/_multigrid.py:223-307
A7 实际执行	运行 C1–C4 的 CPU、MPI size=1 受控实验	确证 本会话命令输出
A8 实际输出	单层达标；转移达标；多层未达 tol；Galerkin 差异可定位	确证 / 未验证 分项标识

来源：分析框架来自 /root/configure/skills/analy/references/analysis-framework.md；A1–A8 已在主体页展开。

分析框架 A9–A15：结果、评价、局限与交接

步骤	本报告结论	证据状态/下一入口
A9 结果分析	C3 说明问题位于粗层 M 的应用分支，而不是 P/R 数值精度	确证 C3 诊断对照
A10 综合分析	算法设计与代码骨架一致，但构造式/应用式断裂会破坏多层数学闭合	推断 由代码 + 实验推出
A11 结尾总结	单层细层求解与转移通过；多层不能宣称已正确	确证 结论先行页
A12 结尾评价	优点是模块化、MPI 内积、NPU 兼容、R3 reset；缺点是上述接口/粗算子风险	确证 / 风险
A13 补充说明	parity、MPI>1、Clover、多 dtype、非二倍粗化均未完成本轮验证	未验证 明确列出
A14 参考源	目标代码、直接依赖、局部 AGENTS、用户伪代码与本会话命令	见来源索引
A15 附录与附件	本报告的 docs/report_multigrid_python_20260828.tex 与同名 PDF、代码直引片段、C1–C4 原始数值与版式账本	确证 交付物

来源：分析框架来自 `/root/configure/skills/analy/references/analysis-framework.md`；A9–A15 的结论均在正文对应页保留来源与状态。

交接索引与版式账本：后续 agent 可直接从证据继续

视觉对象	主张/来源	占用与拆分边界	版式风险/缓解
V1 层级单列表	尺寸递归与输入输出； solver/_multigrid.py:34-45	整页单列，8 行；不拆 循环体	长公式已拆行，列宽约 0.93 倍版心
V2 近零模单列表	$v - A^{-1}Av$ 与 QR； tools/_multigrid.py:8-109	整页单列，8 行；QR 为不可拆单元	公式/中文行均可换行
V3 转移公式	$R = P^\dagger$ 、 $RP = I$ ； tools/_multigrid.py:112-139	左公式 + 右证据，单 页闭合	公式分两式，列宽总和含列 间距小于 0.96
V4 Galerkin 表	构造/应用分支与 C3； dslash/_operator.py:157-226	7 行整页，不拆诊断结 论	长路径使用 pathref，风险 框不跨页
V5/V6 BiCGStab	用户伪代码映射与 R3； solver/_multigrid.py:376-501	两页按“平滑/校正” 语义拆分	单列 p 列，避免 tabular 浮 动体
V7 实测表	C1-C4 原始结果；本会话命令	4 行整页，不拆结果行	数值带科学计数法，未画无 来源曲线
V8 代码直引	源码逐字节片段； solver/_multigrid.py: 376-387, 542-550	每段不超过 12 行	listings 断行；中文正文不 混入代码块

来源：视觉对象、宽高、拆分边界和风险为 report 交接字段；所有主体页均为固定 frame。

附录：代码证据 1——当前层从零初值进入 BiCGStab

```
1     x = torch.zeros_like(b)
2     r = b - matvec(x)
3     r_norm = tools.norm(r)
4     _tol = r_norm*0.5 if level != self.num_level - 1 else r_norm*0.1
5     if self.verbose:
6         print(
7             f"PYQCU::SOLVER::MULTIGRID:\n {level}:Norm of b:{tools.norm(b)}")
8         print(f"PYQCU::SOLVER::MULTIGRID:\n {level}:Norm of r:{r_norm}")
9         print(
10            f"PYQCU::SOLVER::MULTIGRID:\n {level}:Norm of x0:{tools.norm(x)}")
11     if level == 0:
12         self.convergence_history.append(r_norm)
```

读法：代码在 `cycle` 中无条件执行 `x = torch.zeros_like(b)`；因而 `solve` 保存的 `self.x0` 不会改变这一段的初始状态。

来源：原文件直引：`solver/_multigrid.py:376-387`。

附录：代码证据 2——solve 保存 x0，但没有把它传给 cycle

```
1 def solve(self, b: Optional[torch.Tensor] = None, x0: Optional[torch.Tensor] = None) -> torch.  
  Tensor:  
2     if b is not None:  
3         self.b = b.reshape([12]+list(b.shape)[2:]).clone() # sc->e  
4         self.b_list[0] = self.b.clone()  
5     if x0 is not None:  
6         self.x0 = x0.reshape([12]+list(x0.shape)[2:]).clone() # sc->e  
7     self.levels_back()  
8     start_time = perf_counter()  
9     x = self.cycle()
```

读法： x0 在第 547 行被 reshape/clone 到 self.x0，随后直接调用 self.cycle()；cycle 的零初值证据见上一页。

来源：原文件直引：solver/_multigrid.py:542-550。

附录：来源索引、状态与下游动作

来源	用途	状态
<code>solver/_multigrid.py:10-576</code>	主类、V-cycle、停止和接口	代码确证
<code>tools/_multigrid.py:8-139</code>	近零模、局部 QR、转移	代码确证
<code>dslash/_operator.py:150-362</code>	算子组合、粗算子、parity	代码确证; parity 部分未实测
<code>dslash/_wilson.py:10-18, 230-324</code>	Wilson 数学定义与 hopping 实现	代码确证
<code>tools/_linalg.py:7-41、</code> <code>tools/_define.py:126-178</code>	全局内积、范数、布局与 MPI 网格	代码确证
<code>git log --follow --</code> <code>solver/_multigrid.py</code> C1-C4 只读 Python 命令	R2/R3、资源与 seed 演进线索	历史旁证
用户提供的 BiCGStab 表格	运行行为、数值误差、缺陷定位	实测确证/未通过 项
	伪代码排版与早停分支参照; 完整书目信息未提供	外部背景, 非仓库 证据

后续 agent 入口：首先调用 `debug/test` 处理 P0/P1; P0 修复后再用 `pure` 深挖 Galerkin 与 parity 物理映射; 最后调用 `report` 或 `diff` 更新证据与展示。

来源：完整源文件与页码已在各页标注; 本报告未修改任何源代码。

阶段结论

纯 Python 多重网格的层级与 V-cycle 控制流清楚可追踪;
多层正确性当前受粗层 `sitting.M` 应用分支限制。

下一次验证的最小闭环: $RP = I + D_c = P^\dagger D_f P + \|b - Dx\|$ 真残差回归。

来源: 结论汇总自 C1-C4 与 `solver/_multigrid.py` 代码证据。